```
end
```

Given this, we can now add encryption to any model class's attributes using a single call:

```
class Order < ApplicationRecord
  encrypt(:name, :email)
end
```

A small driver program lets us experiment with this:

```
o = Order.new
o.name = "Dave Thomas"
o.address = "123 The Street"
o.email   = "dave@example.com"
o.save
puts o.name

o = Order.find(o.id)
puts o.name
```

On the console, we see our customer's name (in plain text) in the model object:

```
ar> ruby encrypter.rb
Dave Thomas
Dave Thomas
```

In the database, however, the name and email address are obscured by our industrial-strength encryption:

```
depot> sqlite3 -line storage/development.sqlite3 "select * from orders"
     id = 1
user_id =
   name = Dbwf Tipnbt
address = 123 The Street
  email = ebwf@fybnqmf.dpn
```

Callbacks are a fine technique, but they can sometimes result in a model class taking on responsibilities that aren't really related to the nature of the model. For example, in *Participating in the Monitoring Process*, we created